

PROJECT REPORT

Decentralized Healthcare Data Exchange

1. Project Overview

The **Decentralized Healthcare Data Exchange** is a blockchain-based Web3 prototype designed to improve the secure and transparent sharing of healthcare information. It uses Solidity smart contracts to manage user roles, medical-record metadata, patient consent, access permissions, and blockchain audit events. The platform includes four major roles—**Administrator, Hospital, Patient, and Doctor**—with each role having specific responsibilities and permissions within the system.

A key feature of the platform is **patient-controlled access**. Hospitals can add medical-record metadata for registered patients, while patients can decide which doctors are allowed to access their records. Patients can grant access when required and revoke it later, while unauthorized doctors are prevented from accessing restricted records. The project also uses cryptographic hashes to support record-integrity verification and keeps actual healthcare documents off-chain. Synthetic or dummy healthcare data is used throughout the prototype for educational purposes.

2. Problem Statement

Traditional healthcare systems often rely on centralized databases and isolated information systems maintained by individual hospitals, clinics, laboratories, and healthcare providers. This can create **data silos**, making it difficult to exchange medical information efficiently when a patient receives treatment from multiple healthcare organizations. Patients may also have limited visibility and control over who can access their records, while healthcare providers may face delays when obtaining the information required for treatment.

Another major challenge is **security and accountability**. Unauthorized access, accidental modification, or improper sharing of medical information can be difficult to detect in systems where access history is not fully transparent. Conventional systems may also make it difficult to independently verify whether a medical record has been modified after it was created. These challenges highlight the need for a system that provides **controlled access, patient consent, record-integrity verification, and a transparent audit trail**, which forms the basis of the proposed blockchain-based healthcare data exchange.

3. Proposed Solution

The **Administrator** acts as the primary platform manager and is responsible for registering and managing the different participants, including Patients, Doctors, and Hospitals. Once registered, each participant interacts with the system according to their assigned role. **Hospitals** are responsible for adding medical-record metadata for registered patients, such as the record type, file hash, and storage reference, without storing the complete medical document directly on the blockchain.

The **Patient remains the central authority for controlling access to their healthcare records**. A Doctor can access a patient's record only when the Patient has explicitly granted the required permission through the smart contract. The Patient can also revoke previously granted access whenever necessary. Blockchain technology enforces these access rules through smart contracts while providing **consent management, access control, hash-based integrity verification, and an auditable record of important system operations**.

4. Objectives

- **Implement user roles:** Provide separate roles and permissions for Administrator, Patient, Doctor, and Hospital.
- **Manage medical records:** Allow Hospitals to add medical-record metadata for registered Patients while keeping actual files off-chain.
- **Enable patient consent:** Allow Patients to grant or revoke Doctor access to their healthcare records.
- **Ensure secure access:** Prevent Doctors and other unauthorized users from accessing records without valid permission.
- **Verify record integrity:** Use file hashes to detect possible changes or modifications to medical records.
- **Maintain transparency:** Generate blockchain events for important activities such as registration, record creation, access granting, and access revocation.
- **Build a Web3 interface:** Connect the React frontend to Solidity smart contracts using Ethers.js and MetaMask.
- **Demonstrate the system:** Execute and verify the complete healthcare data-sharing workflow on a local Hardhat blockchain.

5. User Roles

- Administrator — registers and manages participants.
- Patient — views records and controls access.
- Doctor — accesses records only when authorized.
- Hospital — adds medical-record metadata.

6. Key Features

- **Role-Based Access Control:** Provides separate permissions and dashboards for Administrator, Patient, Doctor, and Hospital users.
- **Hospital Medical-Record Registration:** Allows registered Hospitals to add medical-record metadata for registered Patients.
- **Patient-Controlled Consent:** Gives Patients control over who can access their healthcare records.
- **Doctor Authorization Checks:** Verifies whether a Doctor has valid permission before allowing access to a Patient's record.

- **Grant and Revoke Access:** Patients can grant access to a Doctor when required and revoke that access at any time.
- **SHA-256 Integrity Verification:** Uses cryptographic hashes to verify whether an off-chain medical record has been modified.
- **Blockchain Audit Trail:** Records important activities such as registration, record creation, access granting, revocation, and record access through blockchain events.
- **User Deactivation and Reactivation:** Allows the Administrator to temporarily deactivate users and reactivate them when required.
- **React, Ethers.js & MetaMask Integration:** Provides a user-friendly React interface connected to Solidity smart contracts through Ethers.js, with MetaMask handling wallet connections and transaction confirmations.

7. Complete Workflow

Administrator The complete workflow demonstrates how healthcare records move through the platform while maintaining **role-based permissions and patient-controlled access**.

Step 1 — Administrator Registration

The Administrator first registers the required participants in the system:

➔ **Patient → Doctor → Hospital**

Each participant is associated with their blockchain wallet address and assigned the appropriate role.

Step 2 — Hospital Adds Medical Record

After registration, the Hospital connects its wallet and adds a medical record for the registered Patient. The Hospital provides the required record metadata, such as the **record type, file hash, and storage reference**.

Step 3 — Patient Views the Record

The Patient connects their wallet and can view the medical records associated with their account along with the available record metadata.

Step 4 — Doctor Attempts Unauthorized Access

The Doctor initially attempts to access the Patient's record without receiving permission.

Doctor → Access Request → Smart Contract → Permission Check → Access Denied

This demonstrates that registration alone does not automatically provide access to patient records.

Step 5 — Patient Grants Access

The Patient selects the required record and grants access to the registered Doctor. The transaction is confirmed through MetaMask and recorded on the blockchain.

Patient → Grant Access → MetaMask → Smart Contract → Permission Granted

Step 6 — Doctor Accesses the Record

The Doctor attempts to access the record again. The smart contract verifies the permission and allows access because the Patient has granted authorization.

Doctor → Access Request → Permission Check → Access Granted

Step 7 — Patient Revokes Access

When access is no longer required, the Patient can revoke the Doctor's permission through the application. The revocation is confirmed through MetaMask and recorded on the blockchain.

Step 8 — Doctor Attempts Access Again

After revocation, the Doctor attempts to access the same record.

Doctor → Access Request → Permission Check → Access Denied

8. On-Chain vs Off-Chain

The project uses a **hybrid on-chain and off-chain storage approach**. On-chain, the blockchain stores information that needs to be trusted, verified, and managed through the smart contract. This includes the **user's role and status, unique record ID, Patient, Doctor and Hospital wallet addresses, record type, file hash, storage reference, timestamp, access permissions, and blockchain audit events**. These details allow the smart contract to identify participants, manage roles, verify access permissions, maintain patient consent, and provide a transparent history of important operations such as record creation, access granting, and access revocation.

The actual healthcare information is maintained **off-chain** rather than being stored directly on the blockchain. This includes complete **medical reports, laboratory reports, prescriptions, vaccination documents, diagnostic records, discharge summaries, and other healthcare documents**. Keeping these files off-chain reduces unnecessary blockchain storage while allowing the blockchain to maintain their important metadata and cryptographic hash. The hash can be used to verify the integrity of an off-chain document by comparing its stored hash with a newly generated hash. Thus, the architecture uses blockchain for **access control, consent, verification, and auditability**, while off-chain storage is used for the actual healthcare documents.

9. Smart Contract Functions

User Management

- **registerUser()** – Registers a new participant in the healthcare data exchange and assigns the appropriate role.
- **deactivateUser()** – Deactivates an existing user, preventing them from performing authorized platform operations.
- **reactivateUser()** – Reactivates a previously deactivated user and restores their permitted platform functionality.
- **getUser()** – Retrieves the stored information associated with a registered user.
- **getUserRole()** – Identifies the role assigned to a particular wallet address, such as Patient, Doctor, or Hospital.
- **isUserActive()** – Checks whether a registered user's account is currently active.

Medical Record Management

- **addMedicalRecord()** – Allows an authorized Hospital to add medical-record metadata for a registered Patient.
- **getRecord()** – Retrieves the metadata and information associated with a specific medical record.
- **getPatientRecords()** – Retrieves the medical records associated with a particular Patient.
- **getRecordCount()** – Returns the number of medical records maintained by the system or associated with the relevant context.

Access Control & Consent Management

- **grantAccess()** – Allows a Patient to grant a Doctor permission to access a specific medical record.
- **revokeAccess()** – Allows a Patient to remove previously granted Doctor access.
- **hasAccess()** – Checks whether a particular Doctor currently has permission to access a Patient's record.
- **getPermission()** – Retrieves the stored permission information for a specific access relationship.
- **accessRecord()** – Performs the authorization check and allows access to a record only when the required permissions are valid.

10. Technology Stack

Frontend

- React
- Vite
- JavaScript / JSX
- CSS
- Ethers.js

Blockchain

- Solidity
- Hardhat
- Ethereum-compatible local blockchain
- MetaMask

Cryptography

- SHA-256

Development

- Node.js
- VS Code
- Git
- Hardhat testing tools

Data

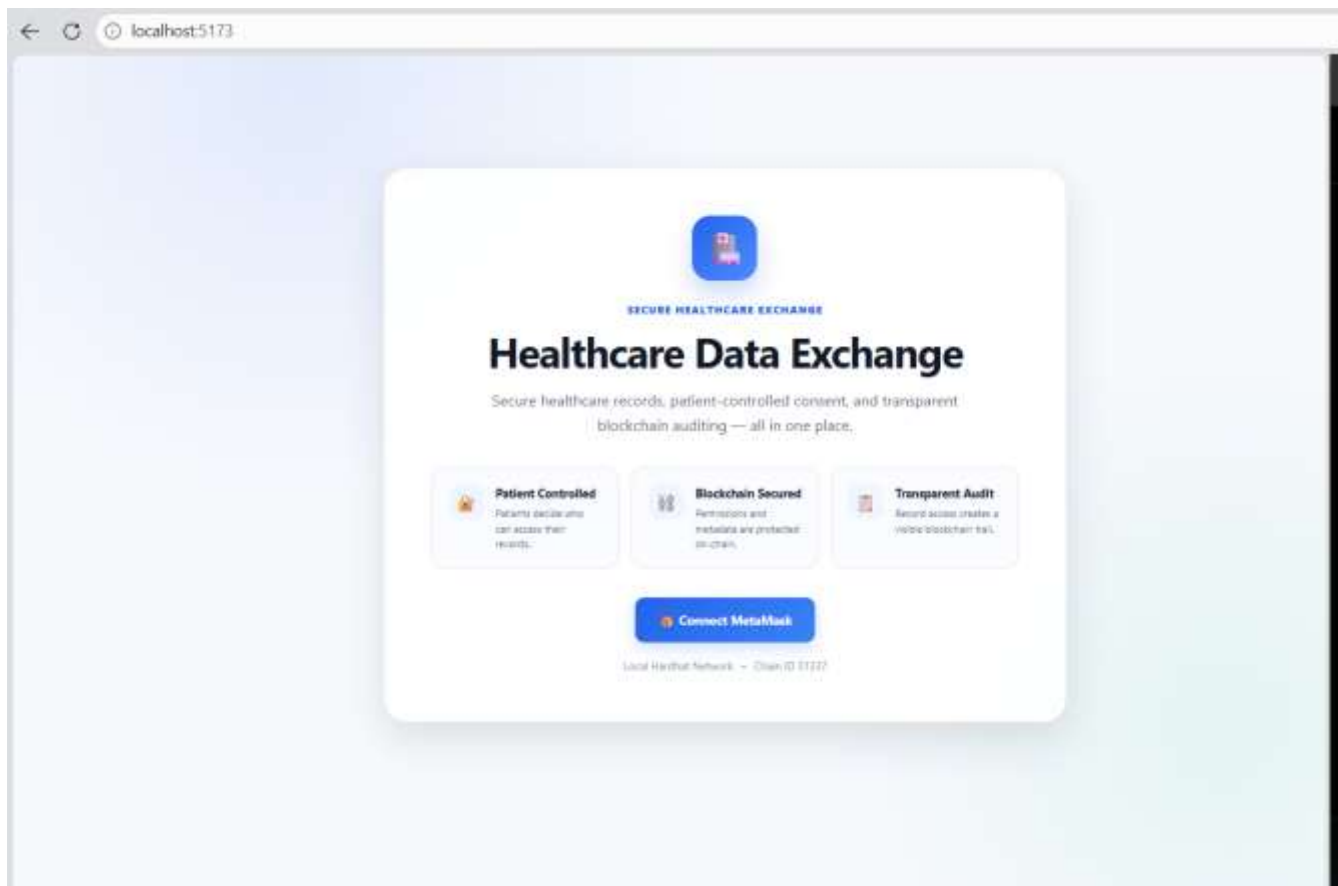
- Synthetic healthcare records
- JSON-based sample medical records
- Off-chain record simulation

11. Testing

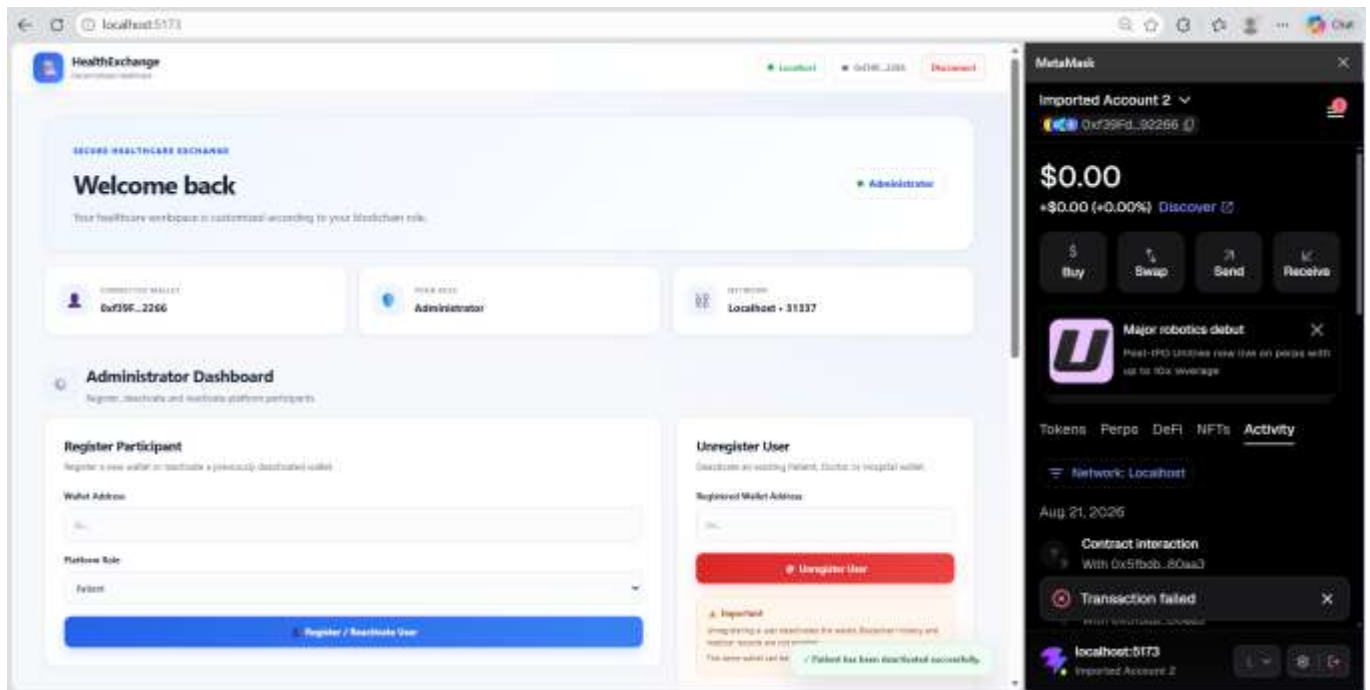
- User registration and duplicate-registration prevention.
- Role validation.
- Medical record creation.
- Unauthorized record creation prevention.
- Unauthorized Doctor access.
- Grant and authorized access.
- Revoke and post-revocation denial.
- Invalid record handling.
- Hash integrity.
- Event emission.

12. Screenshots / Implementation Evidence

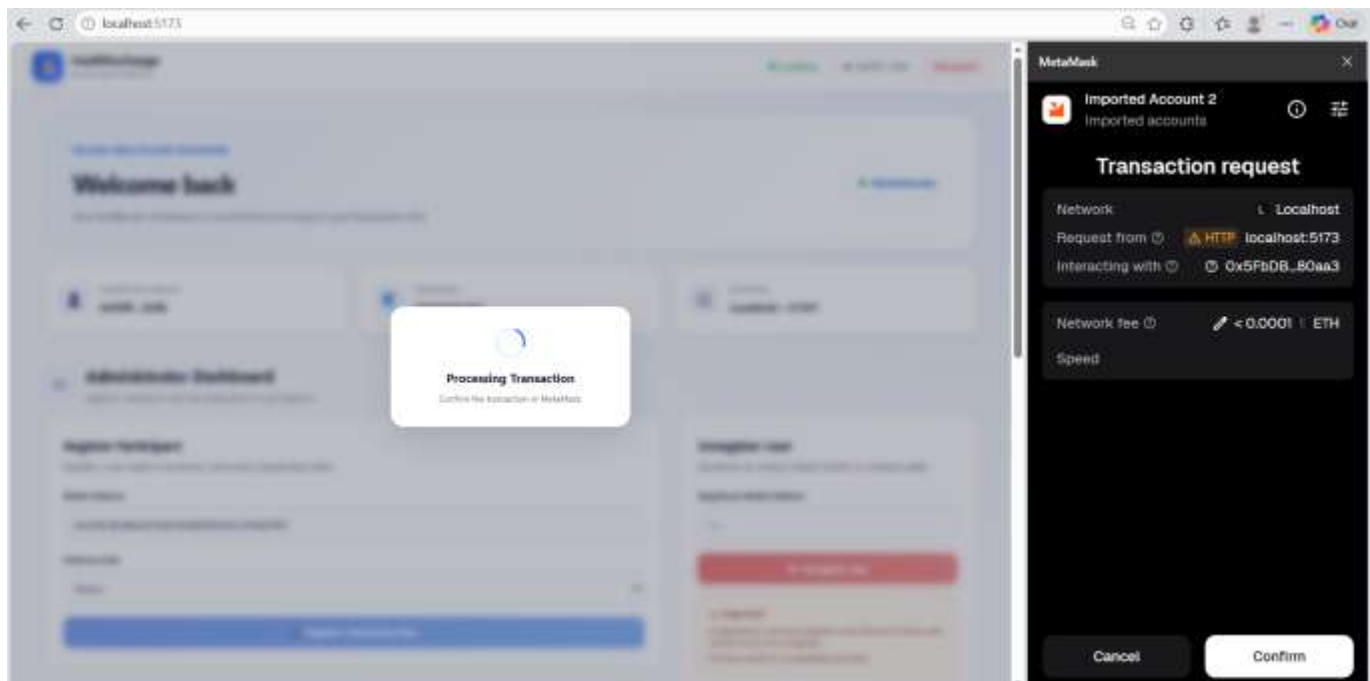
Home Page



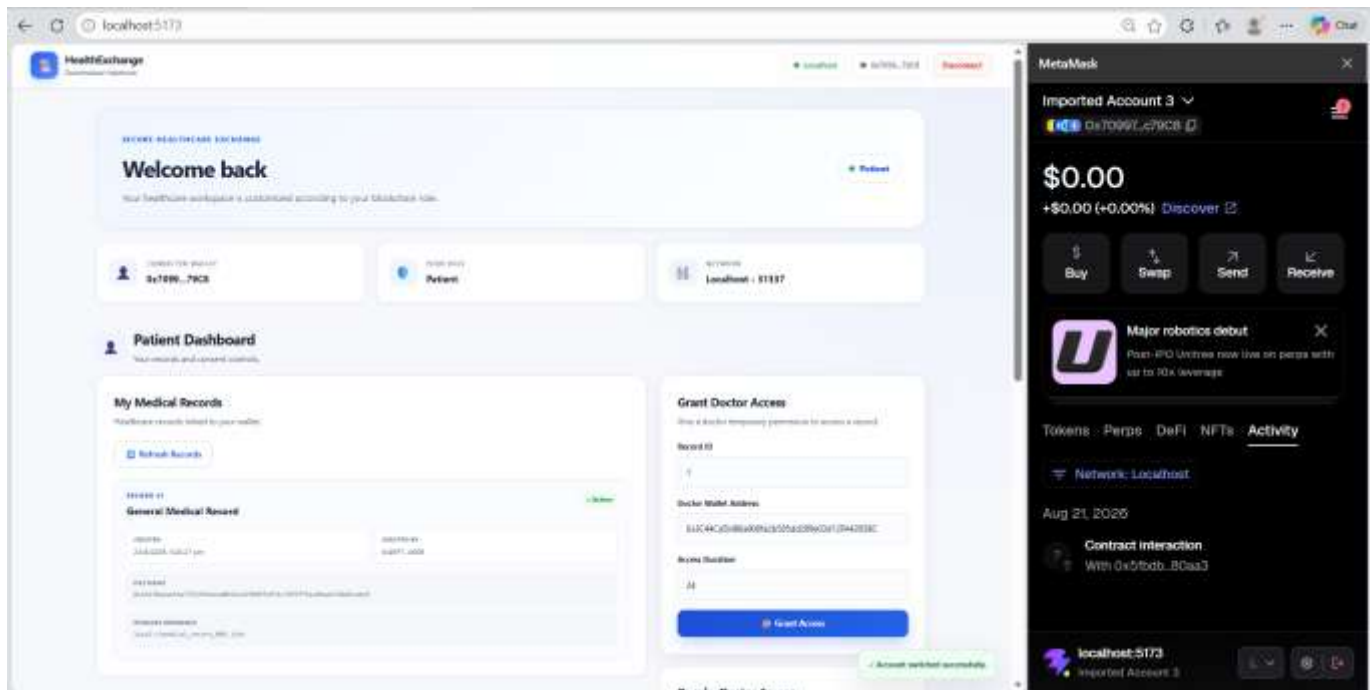
Admin Dashboard



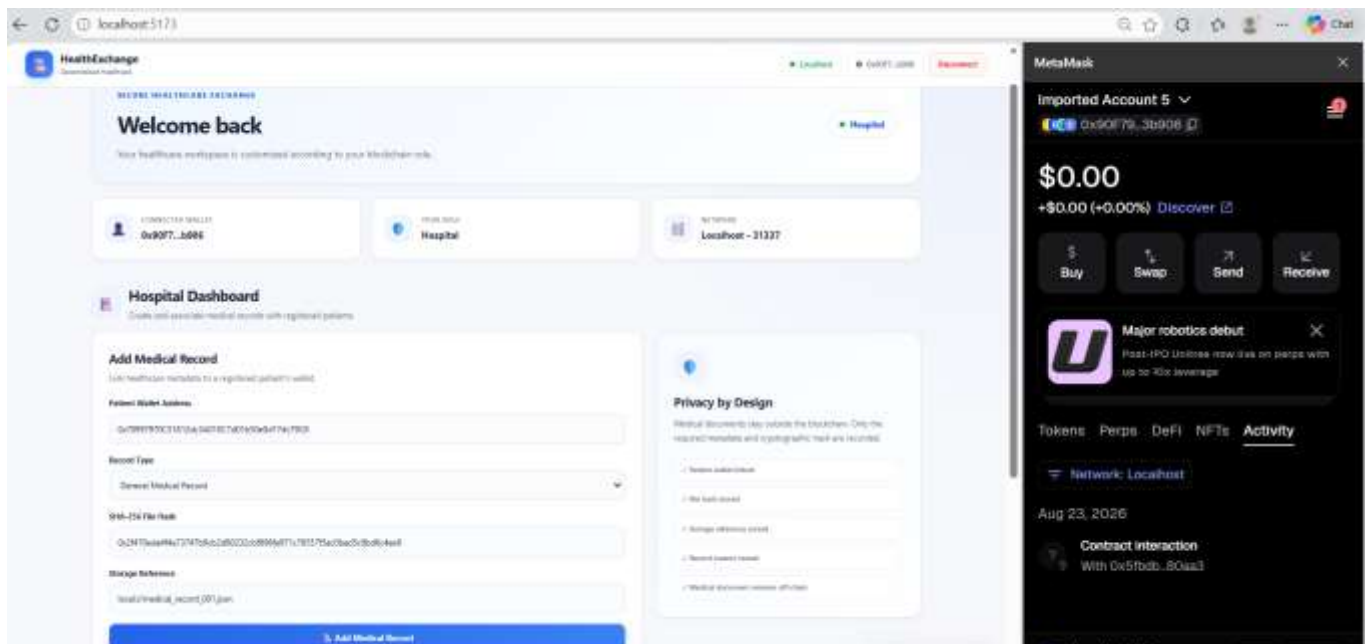
Transaction In Progress



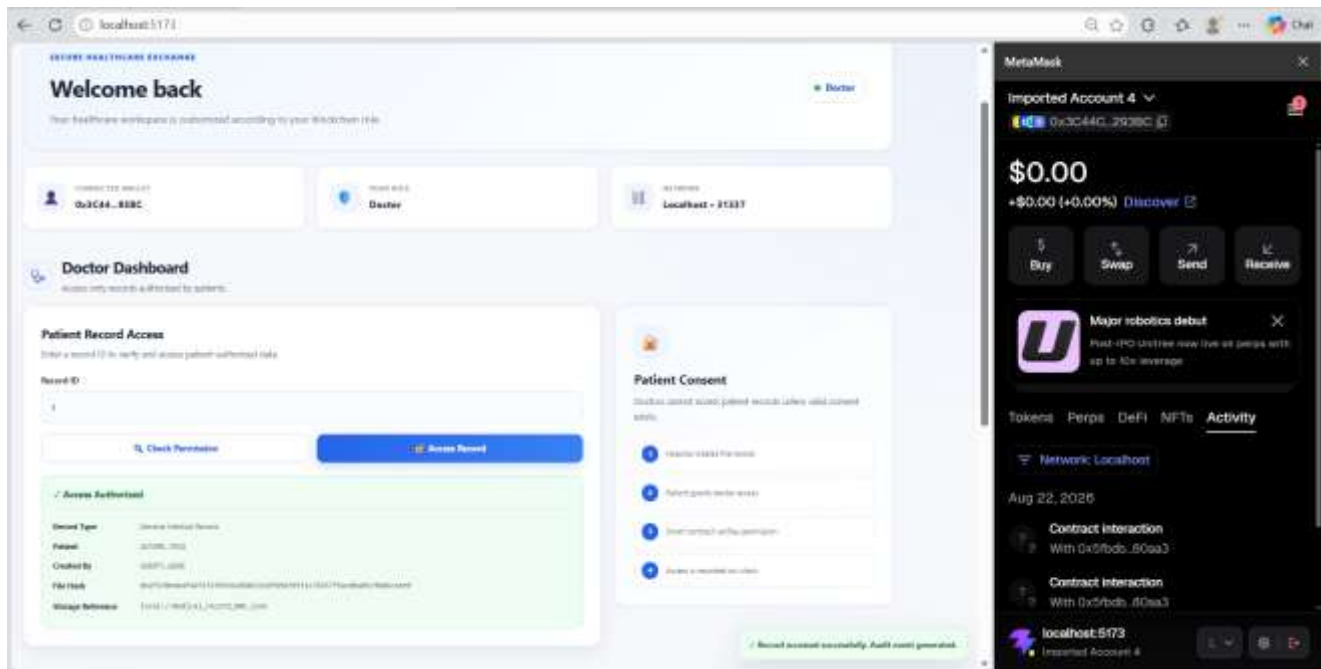
Patient Dashboard



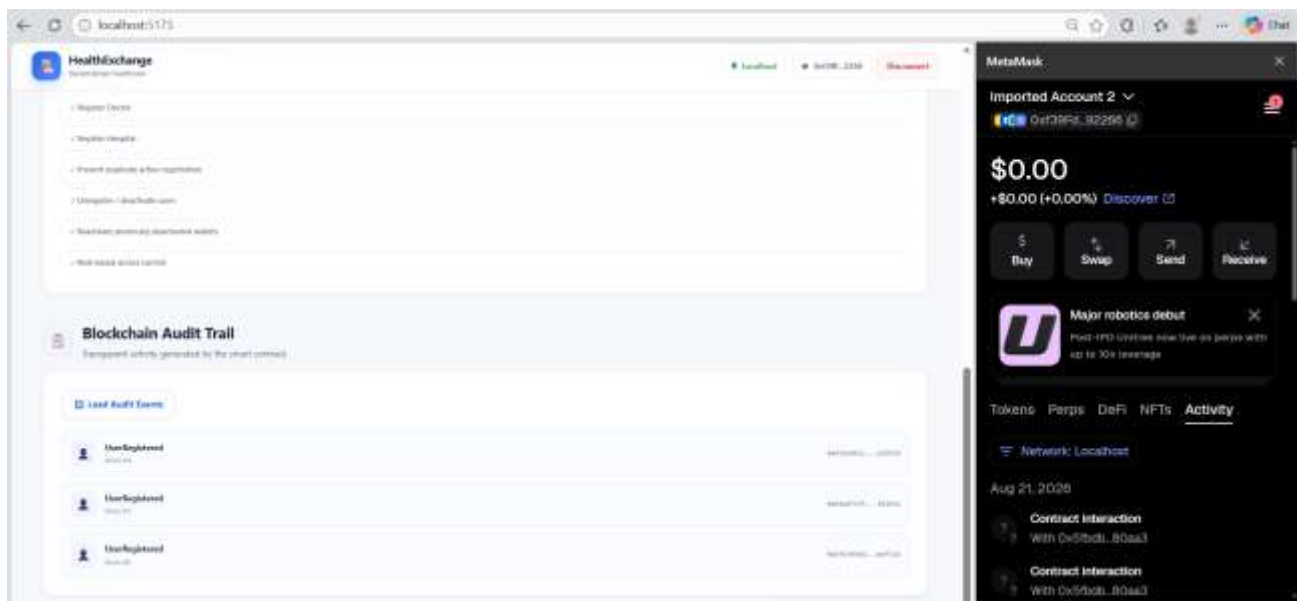
Hospital Dashboard



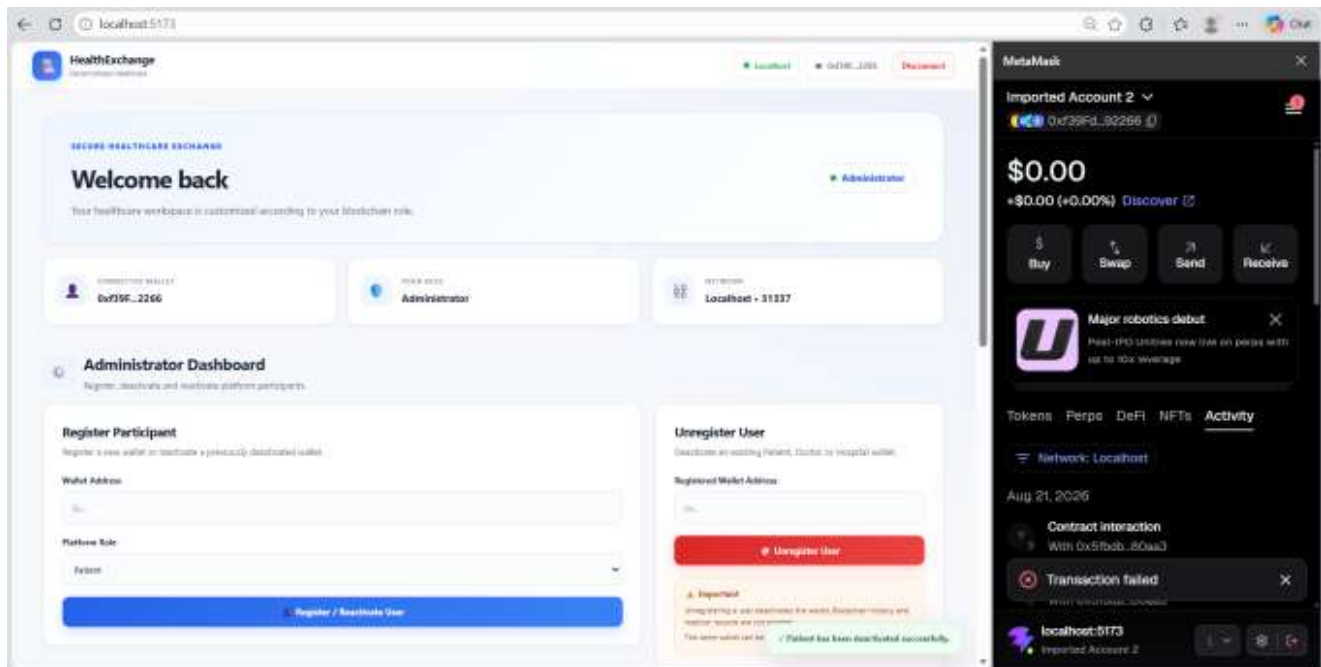
Doctor Dashboard



Events Audit Trail



Patient Deactivation



13. Limitations

- Synthetic healthcare data only.
- Local blockchain development.
- No real hospital/EHR integration.
- Actual medical files simulated off-chain.
- Hashing verifies integrity but is not encryption.
- Revocation cannot delete a copy already downloaded.
- Not intended for production healthcare deployment.

14. Future Improvements

Possible future enhancements include:

- Encrypted off-chain storage
- Time-limited consent
- Purpose-based consent
- Healthcare API integration
- Integration with real healthcare organizations

15. Conclusion

The project demonstrates blockchain as a trust and authorization layer for healthcare data sharing, combining patient-controlled consent, controlled access, integrity verification and auditability.